

James Corino

+1 (845) 699-7699 | jcorino@albany.edu | linkedin.com/in/james-corino | github.com/jamescorino8 | www.jamescorino.dev

EDUCATION

University at Albany, B.S. in Computer Science (Combined B.S./M.S.), Informatics Minor *Expected Dec 2027*

Relevant Coursework: Data Structures & Algorithms, Operating Systems, Computer Architecture, Principles of Programming Languages, Software Engineering

Honors: Dean's List, CNSE Global Scholarship (1 of 2 students awarded full funding for SKKU International Summer Semester)

Sungkyunkwan University (SKKU), Study Abroad | Statistics in Python, AI & Computational Humanities *Summer 2026*

TECHNICAL SKILLS

Languages: C, C++, Java, Python, JavaScript, SQL, HTML/CSS

Frameworks/Libraries: PyTorch, pandas, NumPy, SciPy, Matplotlib/Seaborn, Flask, HTMX, React

Tools: Git, Linux, Docker, Valgrind, JUnit

WORK EXPERIENCE

Sungkyunkwan University *Suwon, South Korea*
Research Intern | Python, PyTorch, Grounding DINO *Jul 2026 – Aug 2026*

- Evaluated Grounding DINO, an open-vocabulary vision-language model, as a face-localization front end for a deepfake-detection pipeline in Prof. Tamer Abuhmed's InfoLab.
- Curated a 40+ image benchmark (occluded, profile, low-resolution, crowded, deepfake frames) and tuned detection thresholds; identified a crowd-density failure mode on small faces (~30–40px) that threshold tuning could not recover.
- Built a Python/PyTorch batch-inference tool with configurable prompts and thresholds, exporting annotated images and JSON/CSV prediction logs with per-image inference times; documented a reproducible environment.
- Delivered weekly reports and a final report and presentation; earned a formal evaluation from the lab director.

Orange-Ulster BOCES *Goshen, New York*
Technical Intern | Python, C, HTMX, Flask *Jan 2024 – Jun 2024*

- Built a full-stack Flask + HTMX prototype exploring decision-making algorithms for interactive design.
- Programmed and debugged microcontroller-based tools in C in a mentored lab environment.

PROJECTS

CPU Simulator | Course Project | Java, JUnit *Feb 2026 – Apr 2026*

- Built a gate-level CPU simulator from scratch: bit logic, 32-bit ALU, ripple-carry adder, barrel shifter, and binary multiplier, all implemented without Java's arithmetic operators.
- Implemented a fetch-decode-execute-store pipeline with 32 registers, a program counter, a call/return stack, and 20+ assembly instructions including branches and load/store.
- Designed a multi-level cache (L1 instruction + L2) with cycle-accurate performance tracking, cutting cycle counts up to 20x on instruction-bound loops and ~74% on memory-intensive workloads (array/linked-list traversal) versus no caching.

Transit Weather Analysis | Team Project | Python, pandas, SciPy, Matplotlib, DeepSeek API *Jul 2026*

- Primary developer on a 3-person team; built an end-to-end pipeline joining Seoul Open Data transit ridership with Open-Meteo historical weather over a 3-month window to test rain's effect on ridership (no significant change, $p = 0.73$).
- Built an LLM-assisted pipeline that scraped and filtered Korean-language YouTube comments on rainy commutes and used the DeepSeek API with structured prompts and validation logic to label sentiment, frustration, and issue type across hundreds of comments; found an association between issue type and rider emotion ($\chi^2 = 21.7$, $p < 0.01$; exploratory given sparse cells).

Personal Portfolio Site | Personal Project | React, Vite, TypeIt, CSS *Mar 2026*

- Designed and built a personal portfolio from scratch in React 19 and Vite, with a custom CSS design system and dark/light theming via CSS custom properties.
- Implemented client-side routing (React Router v7) and a reusable animation hook driving TypeIt typewriter sequences; deployed on Vercel with rewrite rules and analytics.

Nusha Compiler | Course Project | Java, JUnit *Sep 2025 – Nov 2025*

- Built a multi-stage compiler for Nusha, a custom declarative language for logic puzzles: lexer, recursive descent parser, and interpreter for an EBNF-defined grammar, with JUnit tests.

Battleship | Course Project | C, Docker, Valgrind *Sep 2025 – Nov 2025*

- Built a text-based Battleship game in C with single-player and TCP two-player modes; verified Valgrind-clean memory management.